

用于 PageIndex 检索的存储引擎: MongoDB、PostgreSQL、DuckDB 与 SQLite 的工作负载基准

Junyao Dong

摘要。 PageIndex 把文档变成大型分层 JSON 树,智能体从根节点向下导航到相关节点。本报告测量与 ConDB 检索相对应的存储级操作,跨引擎结论只采用对称配置。单独的 300 万节点公平 中,MongoDB 与 PostgreSQL 使用相同 200 条子树路径、返回相同节点 id、并获得 价的覆盖索引。在相同单体布局的 covered arm 上,MongoDB 的 P50/P95 分别慢 4.35/3.91 倍; 方都使用 结构表/集合时,分别慢 4.65/4.07 倍。另一个 MongoDB 单引擎的一千万节点 显示,覆盖索引能做到零文档取出并达到 P50 15.5 ms / P95 168 ms,但该结果不用于跨引擎相除。结构、元数据、正文三路解耦仍是候选 schema;其缓存常驻收益和完整端到端代价 待部署形状的验证。

1 引言

ConDB 把一篇文档索引成一棵 PageIndex 树:节点构成的层级结构,每个节点带有标题、摘要、页码索引,叶子节点还带有正文。检索是一次自顶向下的遍历:检索器从根节点出发,列出某个节点的子节点以决定向哪里下探,拉取一棵子树供语言模型阅读,再获取它所选节点的内容。本项目要回答的问题是:文档型存储(MongoDB)是否适合存放这类数据;如果适合,该优化什么。本报告用实测数据和一份具体方案回答这两点。

我们刻意只对 ConDB 实际发出的操作做基准。检索路径从不执行的通用数据库操作(从节点回溯到根、页码区间过滤、朴素子串扫描)被排除在外,因为它们的结果无助于优化。

2 工作负载与方法

操作

表 1 把所测存储原语映射到 ConDB 检索操作。get_subtree 微基准只测返回节点 id 的物化路径区间扫描,不 根节点读取、元数据解析、树格式化或分页;这些步 被明确列为端到端验证缺 。

操作	存储调用	用途
点查	get_node(tree_id, node_id)	导航到 / 读取单个节点
展开子节点	get_children(tree_id, node_id)	列出子节点,决定下探方向
取子树	get_subtree(tree_id, node_id, depth)	渲染树视图、拉取一个 block
取内容	get_entity(tree_id, node_id)	读取所选节点的内容

表 1:ConDB 检索发出的存储操作。

数据

采用规范格式的合成 PageIndex 树,并用 ConDB 的 DocumentTreeAdapter 校验,两种规模(表 2)。内容是随机词;基准考察的是树的形状与每节点的负载大小。

数据集	节点数	JSON 大小	深度	扇出
中(medium)	70,843	85 MB	5	6-12
大(large)	10,000,000	14.06 GB	8	6-14

表 2:数据集。

引擎与环境

MongoDB 7.0(社区版)与 PostgreSQL 16(JSONB 文档列)以服务端形式跑在 Docker 中、经 localhost 访问;DuckDB 与 SQLite 为嵌入式。原始中/大型 件在 96 核、1 TB

内存主机上使用相同单体布局。跨引擎子树结论只来自另一 的 16 vCPU/30 GiB 主机实验:300 万节点、每次只跑一个引擎、相同 200 条路径(平 返回 8,840.4 个 id)。MongoDB 与 PostgreSQL 都 次测试单体+ 通 path 索引、单体+覆盖索引、 结构 relation+覆盖索引。PG 使用 (path) INCLUDE(node_id) 并 VACUUM ANALYZE;MongoDB 使用 {path,node_id} 且投影排除 _id;两 的 explain 都确认 index-only。一千万节点的 15.5/168 ms 只属于另一 MongoDB 单引擎 。

公平性说明

SQLite 与 DuckDB 在进程内运行,一次查询就是一次函数调用;MongoDB 与 PostgreSQL 要付每次调用的客户端-服务端开销(一次 localhost 往返加协议处理),嵌入式引擎从不承担。因此公平的单次调用比较是 MongoDB 对 PostgreSQL;跨越嵌入式/服务端这条线的绝对微秒数不可比,而并发一节(各自独立进程、无 GIL)展示的是真实的服务端吞吐。MongoDB 的 storageSize 是压缩后的(WiredTiger snappy),其它是未压缩的磁盘文件,所以 MongoDB 的未压缩逻辑大小单独列出;且随机词内容大概比真实散文更易压缩,使压缩后的数字偏好看。每项 比都只在同一 campaign 25 的同一主机内进行;到并发为止的各项用一库一树,多租户、批量读取与删除在下文用 25 棵共存树单独测量。

3 结果

3.1 存储与摄入

引擎	磁盘合计	索引	未压缩	摄入(节点/秒)	建表
DuckDB	5.02 GB	0	—	348,831	10 s
MongoDB	5.81 GB	536 MB	15.21 GB	41,387	69 s
PostgreSQL (JSONB)	17.71 GB	1.19 GB	—	62,816	52 s
SQLite	19.51 GB	980 MB	—	52,880	38 s

表 3:存储与摄入(大数据集)。磁盘合计含索引。MongoDB 以 5.27 GB 磁盘数据(WiredTiger snappy)承载 15.21 GB 逻辑 BSON。

DuckDB 与 MongoDB 最紧凑(表 3),DuckDB 靠列式编码、MongoDB 靠压缩;即便未压缩,MongoDB 的 15.21 GB BSON 也不比关系型引擎的原始表更大。PostgreSQL 与 SQLite 在磁盘上要大上数倍。

3.2 检索操作

导航读取(点查、子节点、取内容)在除 DuckDB 外的每个引擎上都在亚毫秒级,DuckDB 并非为 OLTP 点读而生。在这些读取上 MongoDB 每次调用约比 PostgreSQL 慢 3 倍(表 4),其中一部分是每次调用的客户端开销,但在处处远低于一毫秒的量级上,这点差距没有实际代价。触达大量记录的子树扫描不再与这些单记录读取 表,而只在表 4b 的对称 中比较。covered-to-covered 下,MongoDB 的 P50/P95 分别比 PostgreSQL 慢 4.35/3.91 倍; 方都用 结构 relation 时为 4.65/4.07 倍。第 4 节只解释 MongoDB 内部可 除的 FETCH。

操作	MongoDB	PostgreSQL	DuckDB	SQLite
中数据集,P50				
点查 (get_node)	0.238	0.081	2.705	0.008
展开子节点 (get_children)	0.251	0.090	0.856	0.017
取内容 (get_entity)	0.228	0.071	2.750	0.007
大数据集,P50 / P95				
点查	0.270 / 0.32	0.087 / 0.09	5.59 / 8.97	0.012 / 0.014

展开子节点	0.282 / 0.32	0.099 / 0.14	4.28 / 5.05	0.023 / 0.032
取内容	0.249 / 0.32	0.081 / 0.09	5.21 / 8.35	0.012 / 0.013

表 4a:相同单体布局上的单记录检索延迟(ms,越低越好)。子树扫描不在此表 比。

对称 arm (P50 / P95 ms)	MongoDB	PostgreSQL	Mongo / PG
单体 + 通 path 索引	2.391 / 19.201	0.648 / 6.817	3.69 / 2.82×
单体 + covering index	1.291 / 10.692	0.297 / 2.732	4.35 / 3.91×
structure + covering	1.269 / 10.616	0.273 / 2.610	4.65 / 4.07×

表 4b:公平的 ids-only get_subtree 。300 万节点、相同 200 条路径、平 返回 8,840.4 个 id;每一行都只比较 方相同 arm。

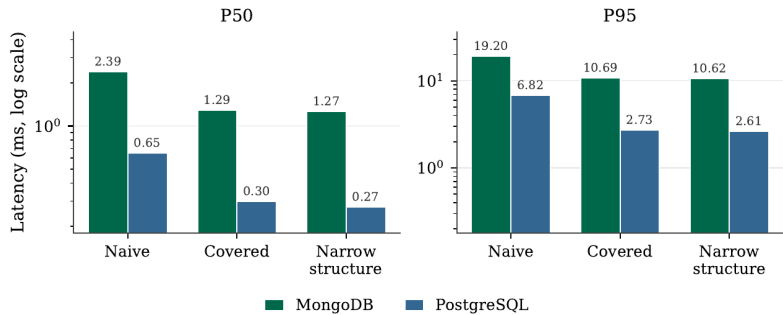


图 1:表 4b 的 MongoDB–PostgreSQL 对称三 结果(对数刻度),不存在优化 Mongo 对未优化 PG。

SQLite 也跑了对称三 ,但它是进程内引擎,不用于服务端主比 。DuckDB 更 :列存投影天然只读 node_id 列,covered 与 naive 按构 价,且没有独立 narrow-structure arm,因此不 加三 排 。

3.3 写入路径

当前设置下,PostgreSQL 的更新延迟低于 MongoDB(表 5),但该实验没有统一 durability 设置,因此只作描 ,不用于公平吞吐排 。5,000 次更新也只能支持“本 跑未测到增长”,不能证明长期永不膨胀。

引擎	更新 P50	更新 ops/s	插入 ops/s	膨胀
MongoDB	0.242	4,050	5,586	0 MB
PostgreSQL (JSONB)	0.123	7,856	8,732	7 MB
DuckDB	2.966	332	450	7 MB
SQLite	0.017	40,442	21,815	0 MB

表 5:写入路径(中数据集):5,000 次字段更新,2,000 次增量插入。durability 未统一,不据此 胜负。

3.4 并发

记录到的单次五秒点查中(图 2),MongoDB 从 1 到 2 客户端先下 、随后一直升到 64;PostgreSQL 的最高值也在 64;SQLite 在 8 个客户端见顶后下 。worker 没有 start barrier,每档也只跑一次,所以本图只作描 性扩展证据,不能当作精确排 或稳定 和点。

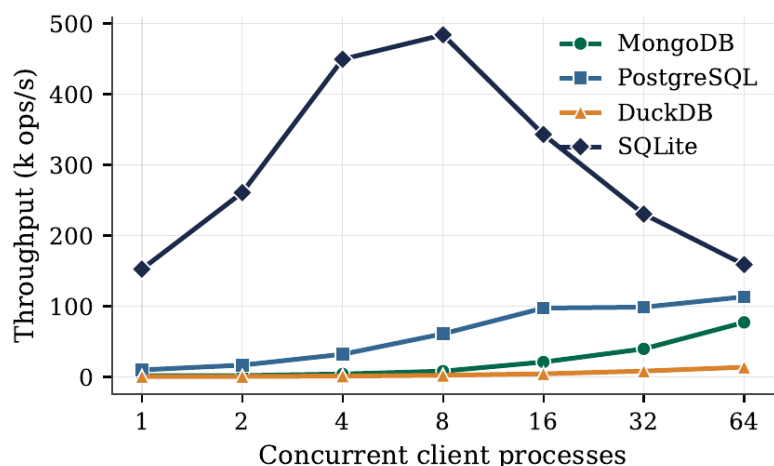


图 2:点查吞吐随并发客户端进程数的变化(中数据集)。

3.5 多租户、批量读取与删除

把检索器的调用映射到存储,会浮现出前几节遗漏的三个操作:多树共存于一个存储时每次查询的租户过滤、导航步发出的批量 id 列表读取,以及整棵树的删除。我们在把中数据集树复制成 25 棵共存树(单一存储内 1,771,075 行)上测量它们,每次读取都按 tree_id 过滤,由以 tree_id 打头的复合索引服务。

多租户选择性。 25 棵树同处一个集合时,以 tree_id 打头的索引让每次读取都贴近其单树延迟(表 6):代价随索引深度变化,而非随共存树的数量。对一个大约 25 倍的存储,MongoDB 在点查与子节点读取上约升 1.5 倍(对照表 4),没有扫描爆炸,且 MongoDB 与 PostgreSQL 的排序不变。

操作	MongoDB	PostgreSQL	DuckDB	SQLite
点查	0.345	0.124	3.772	0.008
展开子节点	0.372	0.147	3.073	0.014

表 6:25 棵树共存于单一存储、各按 tree_id 过滤的每次查询读取(P50,ms)。

批量读取。 导航步返回一组候选节点 id,因此检索器一次取多个节点。一次批量读取(MongoDB 用 \$in、关系型引擎用 IN)在每个引擎上都胜过它所替代的点查循环,且差距随批量增大而扩大,因为在服务端引擎上,循环的每一步都是一次往返。在 200 个 id 时,MongoDB 批量 4.6 ms,而循环 273 ms,差 60 倍;PostgreSQL 是 1.4 ms 对 29 ms(表 7)。MongoDB 受益最大,因为批量摊薄了它在单次读取上所付的每次调用客户端开销;进程内的 SQLite 没有往返可摊,只快约 2 倍。检索器的做法应是用一次批量读取解析候选集,而不是循环。

引擎	10	50	200	循环 / 200
MongoDB	0.46	1.56	4.56	273
PostgreSQL (JSONB)	0.27	0.78	1.40	29
DuckDB	19.6	22.2	39.1	790
SQLite	0.03	0.13	0.50	0.97

表 7:批量 id 列表读取在 10/50/200 个 id 时,对照它所替代的 200-id 点查循环(P50,ms/批)。前三列为批量读取。

删除。 删掉一棵树(此处为一个租户、1,771,075 行中的 70,843 行)是生命周期操作,不属于检索。删除本身在每个引擎上都快(不到四秒),但仅靠删除没有一个引擎会归还磁盘空间(表 8)。SQLite(VACUUM)与 PostgreSQL(VACUUM FULL)会通过一次全量、加锁的重写收回被删树的占用,耗时 10 至 17 秒;MongoDB 的 compact 与 DuckDB 对单租户删除并未让文件明显缩小,这是 WiredTiger 删除后不向操作系统归还空间的既定行为。对 MongoDB 而言这与写入路径对称:WiredTiger 在更新下不背负 vacuum 债,代价则是一次性批量删除不会自动缩容。WiredTiger 以检查点块为粒度报告

storageSize,所以几十 MB 以下的变化是报告粒度,而非已回收的空间。

引擎	删除 行/秒	回收	回收耗时	磁盘(前 → 后)
PostgreSQL (JSONB)	157,578	VACUUM FULL	17.3 s	2.77 → 2.66 GB
SQLite	141,842	VACUUM	10.6 s	2.56 → 2.46 GB
MongoDB	60,680	compact	0.1 s	911 MB,无缩容
DuckDB	20,171	checkpoint	—	790 MB,无缩容

表 8:整树删除与空间回收(在 25 棵树中删除其一)。

这三项都不改变本报告的结论:MongoDB

在大规模下守住租户过滤、在批量读取上完胜,删除上只在一个检索从不触发的、不加锁的缩容上落后。

4 get_subtree 诊断:取出的次数,而非大小

压力是结构性的,而非配置失误。MongoDB 的读取单位是整份文档。一个二级索引(这里在 path 上)是一棵只存被索引字段与记录 id 的独立 B 树;要返回任何其它字段,引擎都必须取出完整的 BSON 文档再对其投影。一次点查就是一次这样的取出,很便宜;在单体节点文档——结构、标题、摘要、叶子正文都在一份文档里——上扫一棵大子树,则是数万次,且无论投影多小,每次都读出并解码整份文档。我们向 MongoDB 提出这一点时,其产品团队确认了该读取路径,并确认新版服务端(含 8.0 SBE)不改变它。数据常驻缓存(且 WiredTiger 缓存中的页是未压缩存放的),所以代价既非磁盘 I/O 也非解压:时间花在每文档的工作上——定位记录、物化文档、应用投影、经游标传出——再乘以子树大小。其它检索操作只触达少量文档,在任何布局下都快。

两组受控测量(单客户端、集合常驻、同一组 200 条子树路径)把代价钉在取出的次数而非大小上。其一,16、32、64 个后台进程对同一服务端狂打点查时,单体布局的扫描延迟几乎不动,全程零缓存逐出:代价就是扫描本身,不是争用。其二,缩小被扫描的文档没有用。把宽 text 字段拆出去,每份被扫文档缩小约 75%,子树延迟不变;哪怕只剩约 150 字节的纯拓扑文档,取出成本的大头仍在——定位并物化约 3.6 万份文档的开销与它们的重量无关。因此只有去掉取出这一步才有用,这正是候选 schema 做的事(第 5 节)。

行式与列式引擎在每个节点上做的工更少。行存访问每一行匹配,却只物化被请求的列;宽 text 留在行里未被读取(PostgreSQL 甚至不解压该 JSONB datum,因为 node_id 是独立的列)。列存只以向量化批次读取被请求的那一列。这是文档模型的取舍,也是 MongoDB 在别处取胜的原因:把一个节点的字段存在一起,让“取出整个实体”很快(点查与子节点读取)、让就地更新便宜(无 vacuum 债)、让磁盘形态紧凑。PageIndex 工作负载在除大子树扫描之外的每个操作上都顺着这个模型——因此候选 schema 让这个操作改由索引服务。

5 候选 schema:结构解耦 + 键值元数据

一个节点携带三类数据,检索在三条不同的路径上读它们,候选 schema 就把三者分开存:

结构(node_id、path、parent_id、depth):约 150 字节的拓扑文档,带精简覆盖索引 {path, node_id}(生产中以 tree_id 打头),服务 get_subtree 与导航。

元数据(title、summary):以 (tree_id, node_id) 或 价 namespace 的 _id 为键。

正文:叶子正文独立成集,使用同样的租户限定键,服务 get_entity。

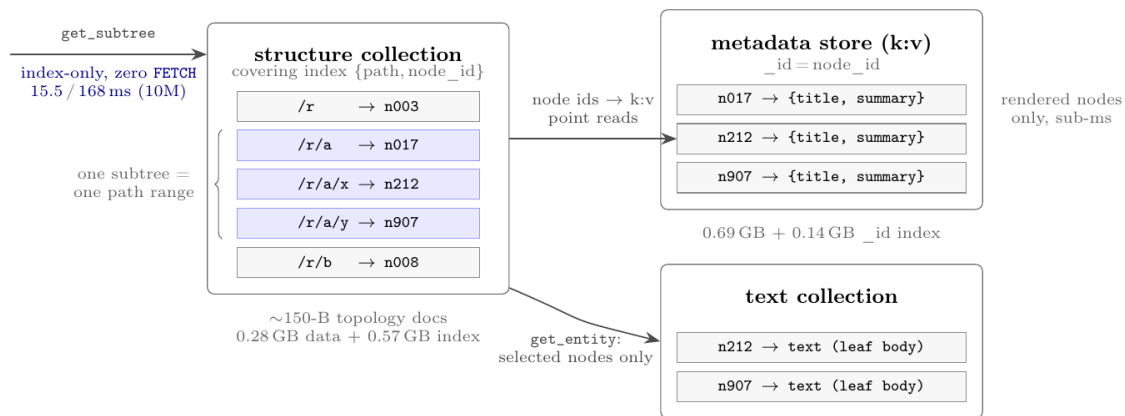


图 3:候选 schema。一棵子树就是小覆盖索引 {path, node_id} 上的一段连续区间,get_subtree 全程不离开索引;它返回的 node id 即元数据存储(k:v)与正文集合的键,只为实际渲染或被选中的节点读取。

在此布局下,子树扫描不再触碰任何文档:explain 确认 PROJECTION_COVERED、totalDocsExamined: 0,一千万节点树上 get_subtree 为 P50 15.5 ms / P95 168 ms(表 9),这是 MongoDB 单引擎绝对值,不与其它 campaign 的 PostgreSQL 数字相除。MongoDB 产品团队提示了两个陷阱:覆盖执行是全有或全无——索引漏掉任何被投影字段、或投影里留着 _id,每文档取出就悄悄回来(结构集合不走覆盖索引时,哪怕文档只有约 150 字节,也要慢约 1.6 倍);覆盖索引的维护成本落在写入上,而写一次的树把它变成一次性的摄入成本——先批量装载,再建索引。

读取	P50	P95	说明
锚点:去 text 单体布局,视图(FETCH)	38.3	421	对齐此前实验
锚点:去 text 单体布局,id(覆盖)	17.3	181	对齐此前实验
结构扫描,id,无覆盖索引	25.2	260	每节点一次 FETCH
结构扫描,id(覆盖)	15.5	168	totalDocsExamined: 0
+ 全部约 36k 节点元数据,1k-id 分块	96.3	1034	harness:约 36 次调用

表 9:MongoDB 单引擎的一千万节点 schema (单客户端,ms),不是跨引擎证据。

ids-only 扫描不能代表完整树视图。元数据行一次解析约 3.6 万节点,约 1 秒 P95,但约 36 次调用来自 harness 固定的 1,000-id 分块,不是 16 MiB 上限;当前 id 的完整 BSON 命只有约 0.62 MiB。该行是机制压力测试,不是生产端到端延迟。若部署确实需要单条索引查询返回完整视图,备选是把元数据并入合并集合的覆盖索引:它仍然被覆盖、胜过单体扫描,但数 KB 的摘要键会把索引推到与数据本身相当的体积——这正是候选 schema 把元数据放进键值存储的原因。

拆分同时让热工作集很小。一千万节点的结构集合在磁盘上是 0.28 GB 数据加 0.57 GB 索引(WiredTiger 前缀压缩去重了共享的 path 前缀),元数据存储再加 0.69 GB 与 0.14 GB 的 _id 索引:导航工作集合计约 1.7 GB,而含正文的单体集合是 5.3 GB。生产规模上,多树多租户共享主机时,这一差距决定导航字节能否常驻缓存:正文永远不与结构争抢 WiredTiger 缓存。

更大的收益需要结构化反规范化(未实测)。Nested Sets 或按树序 _id 的 clustered collection 把子树变成一次区间查询或顺序读;预计算的子树文档把它变成一次点读。三者都因树基本写一次而可行(代价在装载时一次付清),也可能低每键扫描成本。但我们没有测它们,因此不能声称会平 PostgreSQL;是否值得增加 schema 复杂度必须由完整端到端测量决定。(\$graphLookup 不是候选:它重走 parent_id、仍然物化文档,7.0 里还有 100 MB 上限且无视 allowDiskUse。)

6 部署验证方案

在缓存常驻的主机上测得;生产部署还要加上相反的约束——多树多租户可能让工作集大于内存,方案因此是:

1. 端到端验证解耦 schema。
结构(tree_id、node_id、parent_id、depth、path)进导航集合;title 与 summary 进以 (tree_id, node_id) 为键的元数据集合;正文同键另存。表 9 只近 其中的 MongoDB covered scan;完整三集合读取与格式化 未测量。

2. 每个索引与分片键都以 tree_id 打头。
多租户读取必须保持租户局部:点查、子节点、子树区间、取内容都应由 tree_id 打头的复合索引服务。树的数量增长时选择性不衰减,也是水平扩展的天然分片键。

3. 覆盖索引保持精简、并与投影严格对齐。{tree_id, path, node_id} 让子树扫描仅凭索引运行;批量装载后再建,维护成本一次付清。覆盖执行全有或全无——漏一个被投影字段、或投影留着 _id,每文档 FETCH 就悄悄回来——所以把 explain 显示 totalDocsExamined: 0 当作部署检查项,而非一次性的观察。不要把数 KB 的摘要放进覆盖键:元数据存储已经按点查服务它,放进索引要付出与数据相当的索引体积。

4. 只有在实测出需求后才反规范化热子树视图。 若 get_subtree 仍是主要延迟来源,用写一次的结构预计算有界的结构+摘要子树视图,或加 Nested Sets / 树序 _id 换取区间局部性。它们能去掉覆盖扫描剩余的每键工作,但带来存储放大与更复杂的摄入。

上线验证应复现本基准缺失的生产条件:把 WiredTiger 缓存压到单体内联布局的工作集之下、解耦导航工作集之上,对照两种布局。那能直接给拆分的缓存常驻收益定价;本基准做不到,因为它测的每种布局都已常驻缓存。

7 结论

基准 别出一个有 的 MongoDB 优化,而不是跨引擎持平。对称 300 万节点 中,覆盖索引把 MongoDB 的 get_subtree 从 P50/P95 2.391/19.201 ms 至 1.291/10.692 ms;PostgreSQL 获得 价优化后为 0.297/2.732 ms,因此 MongoDB 仍慢 4.35/3.91 倍。 方都用 结构 relation 时,MongoDB 仍慢 4.65/4.07 倍。这些只是公平的 ids-only 微基准比 ,不是完整端到端比 。

设计背后的诊断是机制性的、且经厂商确认:MongoDB 读整份文档,未覆盖的扫描按命中文档数计费,与投影大小无关,新版服务端与缓存调优都不改变它。解耦让扫描路径上只剩扫描需要的字节(覆盖索引下的结构),一千万节点的 15.5/168 ms 只保留为 MongoDB 单引擎容量点,不与 PostgreSQL 数字相除。剩余验证 受限缓存 A/B,以及使用真实 formatter、租户限定键和结果 界的完整 structure/metadata/text 路径。